

QualiVision AI

AI-Powered Industrial Quality Control System

A Final Project Report

Submitted by:

Jiphin George

Course:

Master of Computer Applications (MCA)

Internship Organization:

Nestsoft Technomaster

GitHub Repository:

<https://github.com/JiphinGeorge/SmartCam-AI---Industrial-Quality-Control-System>



Date: July 2026

Declaration

I hereby declare that this project report entitled "QualiVision AI – Industrial Quality Control System" submitted in partial fulfillment of the requirements for the Master of Computer Applications (MCA) is an authentic record of my own original work carried out during my internship at Nestsoft Technomaster.

The matter embodied in this report has not been submitted by me for the award of any other degree or diploma to any other University or Institution.

Place: Ernakulam, Kerala

Date: July 2026

Signature:

(Jiphin George)

Acknowledgement

I would like to express my deepest gratitude to Nestsoft Technomaster for providing the opportunity to work on this capstone internship project. I am also grateful to my mentors, professors, and colleagues for their continuous support, guidance, and technical insights which made the successful completion of QualiVision AI possible.

Abstract

Manual quality inspection on industrial manufacturing and sorting lines is often slow, inconsistent, and labor-intensive. To overcome these limitations, this project introduces QualiVision AI, an enterprise-grade, real-time computer vision system tailored for industrial quality control. Built using a robust Model-View-Controller (MVC) architecture, it integrates a Flask backend with a Deep Learning pipeline powered by TensorFlow and EfficientNetV2B0.

The system processes real-time camera feeds to perform automated anomaly detection (such as sorting fresh vs. rotten agricultural produce). It features Grad-CAM visualizations for Explainable AI, persistent SQLite event logging, and WebSocket-powered telemetry for live dashboard analytics. The solution achieved a 99.82% validation accuracy with sub-second inference times, demonstrating significant potential for automating physical quality assurance tasks, reducing costs, and increasing production throughput.

Table of Contents

AI-Powered Industrial Quality Control System	1
Declaration	2
Acknowledgement	3
Abstract	4
Table of Contents	5
List of Figures	7
List of Tables	8
Abbreviations	9
1. Introduction	10
2. Problem Statement	10
3. Objectives	10
4. Scope of the Project	10
5. Existing System	11
6. Proposed System	11
7. System Requirements	11
7.1 Hardware Requirements	11
7.2 Software Requirements	11
8. Technology Stack	11
9. Project Architecture	12
10. Database Design	12
11. Dataset	13
12. AI Model	14
13. Prediction Pipeline Workflow	15
13.1 Handling Out-Of-Distribution Data (Unknown Class)	15
14. Flask Backend & APIs	15
15. Application Modules	16
15.1 Executive Dashboard	16
15.2 Live Monitoring	17
15.3 Inspection Studio	18
15.4 Analytics	20
15.5 Intelligent Reporting	21
15.6 Inspection History	21

15.7 Dataset Repository	22
15.8 Model Management	23
15.9 Knowledge Center	23
15.10 Settings.....	24
16. Testing & Validation	24
17. Performance Analysis	25
17.1 Confusion Matrix.....	26
18. Project Timeline & Challenges.....	27
18.1 Timeline	27
18.2 Challenges Faced	27
19. Industrial Applications	27
20. Future Scope.....	27
21. Conclusion	28
22. References.....	28
23. Appendix	28
23.1 Folder Structure	28
23.2 Requirements (requirements.txt).....	28
23.3 Database Tables Schema.....	29
23.4 API Sample JSON Response	29
23.5 Code Availability	29

List of Figures

Figure 1: System Architecture Block Diagram	12
Figure 2: Entity-Relationship Diagram	13
Figure 3: Prediction Pipeline Flowchart	15
Figure 4: Executive Dashboard Module.....	17
Figure 5: Live Camera Monitoring Module.....	18
Figure 6: Animated Inspection Studio & Grad-CAM.....	19
Figure 7: System Dark Mode and Out-of-Distribution Views	20
Figure 8: Performance Analytics Module	20
Figure 9: Intelligent Reporting Module	21
Figure 10: Inspection History and Log Search	22
Figure 11: Dataset Repository Management	22
Figure 12: AI Model Management	23
Figure 13: Knowledge Center Documentation	24
Figure 14: Global Settings and Configuration	24
Figure 15: Training Performance over 25 Epochs	26
Figure 16: Validation Confusion Matrix.....	26

List of Tables

Table 1: List of Abbreviations	9
Table 2: Technology Stack Details	11
Table 3: Dataset Train / Validation / Test Split.....	13
Table 4: AI Model Training Hyperparameters	14
Table 5: Core API Endpoints	16
Table 6: System Testing Results	25
Table 7: System Performance Metrics.....	25

Abbreviations

Table 1: List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
CNN	Convolutional Neural Network
ER	Entity-Relationship
Grad-CAM	Gradient-weighted Class Activation Mapping
LOF	List of Figures
LOT	List of Tables
MVC	Model-View-Controller
OOD	Out-of-Distribution
QC	Quality Control
RBAC	Role-Based Access Control
REST	Representational State Transfer
TOC	Table of Contents

1. Introduction

In the modern era of Industry 4.0, automation and intelligent systems play a critical role in increasing efficiency and reducing operational overhead. One of the most challenging areas in manufacturing and agricultural processing is Quality Control (QC). QualiVision AI was developed as a comprehensive solution to automate this process.

The application consists of 11 dedicated modules integrated into a unified industrial dashboard using Google Stitch-inspired UI/UX. Using state-of-the-art Deep Learning models, the application processes images captured from a factory line and predicts whether an item passes or fails quality standards. It then surfaces this data to an enterprise dashboard, providing operators with actionable insights, detailed analytics, and explainable AI metrics.

2. Problem Statement

Manual quality inspection on modern industrial manufacturing and sorting lines is typically:

- Slow: Cannot keep up with the physical throughput of conveyor belts.
- Inconsistent: Prone to human error, visual fatigue, and subjective judgments.
- Labor Intensive: Requires dedicated, round-the-clock staffing for monotonous tasks.
- Expensive: Results in high operational costs over time.

3. Objectives

- To develop an automated vision system capable of identifying defective objects in real-time.
- To provide Explainable AI (XAI) using Grad-CAM so operators can trust the model's decisions.
- To build an industrial dashboard for real-time monitoring and telemetry.
- To implement persistent data logging and automated reporting for production managers.

4. Scope of the Project

The scope encompasses the development of the AI model, the backend REST API, real-time WebSocket communication, and the frontend web dashboard. It focuses on a 2-class classification problem (e.g., Fresh vs. Rotten) but is architected to scale to multi-class defect detection. It is designed to be deployed on local factory edge servers rather than cloud environments to ensure minimal latency.

5. Existing System

Current systems either rely entirely on manual human inspection or use primitive optical sensors (like infrared or basic color thresholding). These legacy systems struggle with complex organic shapes, varying lighting conditions, and nuanced defects, leading to high false-positive and false-negative rates.

6. Proposed System

QualiVision AI proposes a Deep Learning approach using Convolutional Neural Networks (CNNs). By training on thousands of diverse images, the model learns complex features. Integrated with a web platform, it offers 24/7 constant analysis, sub-second inference, and detailed visual reporting.

7. System Requirements

7.1 Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or higher
- RAM: 8 GB Minimum (16 GB Recommended)
- GPU: Dedicated NVIDIA GPU (e.g., RTX 3050) with CUDA support for inference
- Storage: 500 MB for application, additional space for datasets

7.2 Software Requirements

- OS: Windows 10/11, Ubuntu 20.04+, or macOS
- Python: 3.10+
- Database: SQLite 3

8. Technology Stack

As shown in Table 2, the system utilizes a robust web and machine learning stack:

Table 2: Technology Stack Details

Layer	Technology
Frontend	HTML, CSS (Tailwind), JS
Backend	Flask, Flask-SocketIO
AI / ML	TensorFlow, Keras
Model	EfficientNetV2B0
Database	SQLite

Vision / Camera	OpenCV
-----------------	--------

9. Project Architecture

QualiVision AI utilizes a robust Model-View-Controller (MVC) architecture powered by Flask Blueprints and Socket.IO. The client accesses the Dashboard UI over HTTPS, which communicates via WebSockets for live video streams and telemetry, and REST APIs for static data operations. The Flask backend handles security and routes images to the TensorFlow Inference engine, which predicts results, generates a Grad-CAM heatmap, and logs the event to SQLite.

As shown in Figure 1, the architecture flows from the physical camera directly into the dashboard.

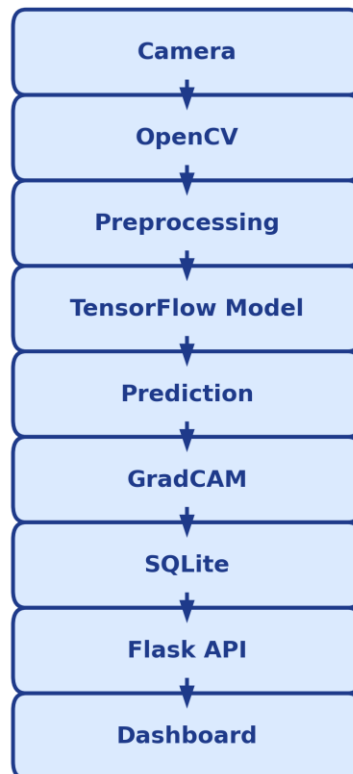


Figure 1: System Architecture Block Diagram

10. Database Design

The SQLite database uses a relational structure linking users to their inspection activities and system configurations.

As shown in Figure 2, the database ensures referential integrity between users and predictions.

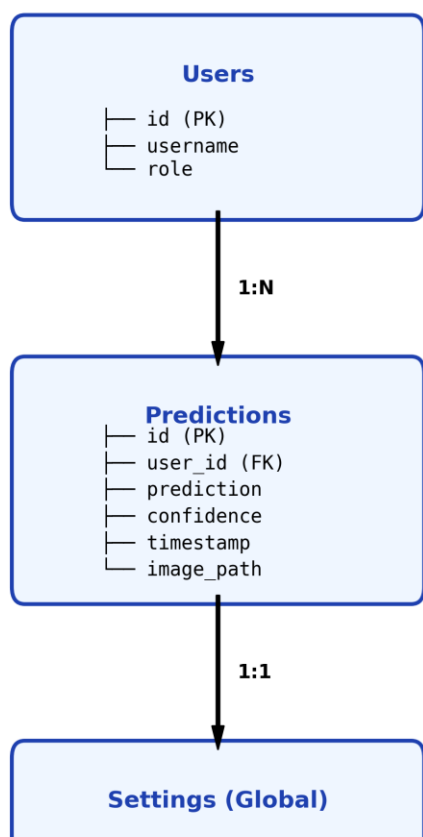


Figure 2: Entity-Relationship Diagram

11. Dataset

The neural network is trained on a rigorously prepared and audited agricultural dataset collected from open sources.

Dataset Sources: Aggregated from Roboflow and Kaggle open datasets.
 Augmentation techniques: Random rotation, horizontal flipping, contrast adjustments, and zooming to prevent model overfitting.

As shown in Table 3, the dataset is perfectly balanced across classes.

Table 3: Dataset Train / Validation / Test Split

Split	Images
Train	5530

Validation	1185
Test	1185
Total	7900

12. AI Model

The core vision model is EfficientNetV2B0, known for its excellent trade-off between accuracy and computational efficiency. Transfer learning was utilized, starting with ImageNet weights and fine-tuning the dense layers for the binary classification task.

As shown in Table 4, the model is highly optimized for fast inference.

Table 4: AI Model Training Hyperparameters

Parameter	Value
Model Summary	EfficientNetV2B0 Feature Extractor + Dense Head
Number of Parameters	7,189,235
Frozen Layers	260
Trainable Layers	8 (Dense and BatchNormalization)
Image Size	224 x 224 RGB
Batch Size	32
Epochs	25
Optimizer	Adam
Learning Rate	1e-4
Loss Function	Binary Crossentropy
Validation Accuracy	99.82%
Validation Loss	0.0076
Training Time	~45 minutes (RTX 3050)

13. Prediction Pipeline Workflow

As shown in Figure 3, images pass through a rigorous pipeline before being surfaced to the dashboard.

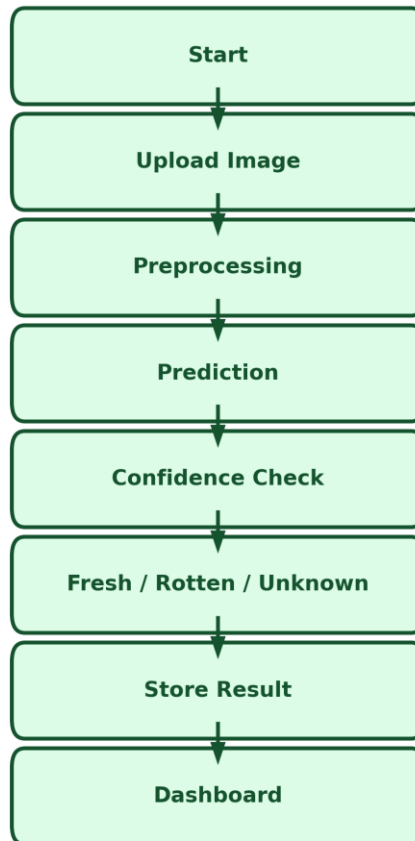


Figure 3: Prediction Pipeline Flowchart

13.1 Handling Out-Of-Distribution Data (Unknown Class)

To prevent the AI from making erratic guesses on non-related objects, a confidence threshold logic is strictly enforced. If an object does not confidently belong to the trained classes, it is rejected to a manual inspection queue:

```
IF Fresh Probability > Threshold (65%) THEN Class = Fresh  
ELSE IF Rotten Probability > Threshold (65%) THEN Class = Rotten  
ELSE Class = Unknown -> Route to Manual Inspection Required
```

14. Flask Backend & APIs

The backend is structured using Flask Blueprints to separate concerns. REST APIs power the dynamic nature of the app.

As shown in Table 5, REST principles are strictly adhered to.

Table 5: Core API Endpoints

Endpoint	Method	Purpose
/api/predict	POST	Execute inference on uploaded image
/api/history	GET	Fetch, filter, and search inspection history logs
/api/stats	GET	Retrieve real-time dashboard telemetry and statistics
/api/report	GET	Generate and export PDF/CSV analytical reports

15. Application Modules

15.1 Executive Dashboard

The Executive Dashboard acts as the central nerve center for the QualiVision AI system. It provides high-level telemetry, displaying total inspections, pass/fail rates, and live activity streams. The UI utilizes dark mode elements and glassmorphism to present critical industrial data in a clean, modern way without overwhelming the operator.

As shown in Figure 4, the module offers a bird's-eye view of operations.

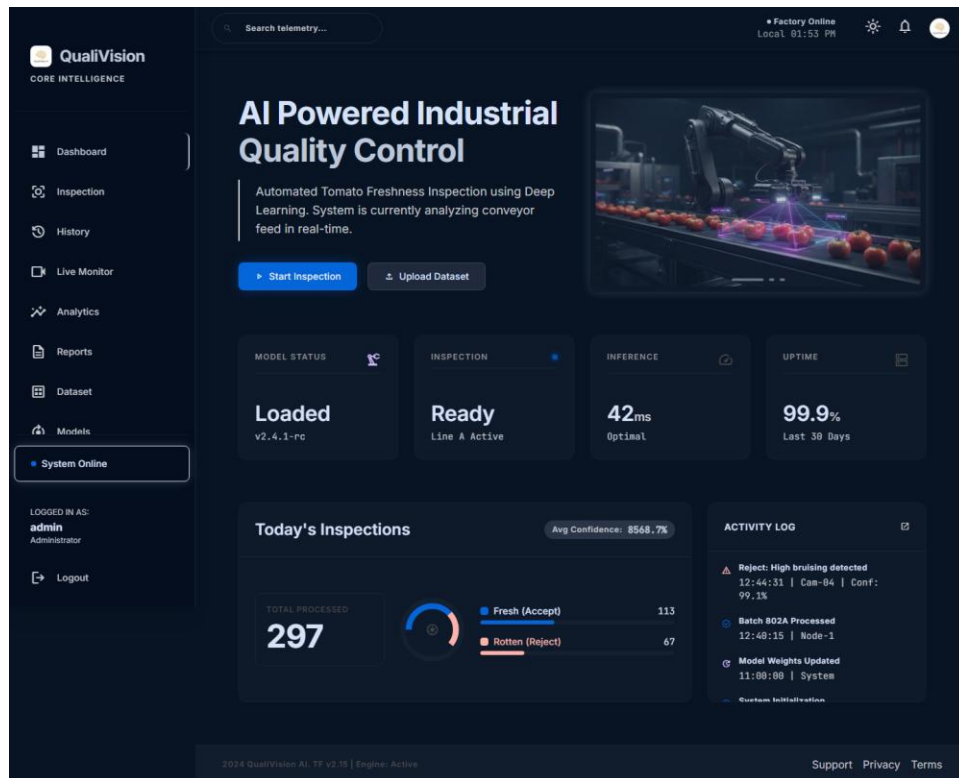


Figure 4: Executive Dashboard Module

15.2 Live Monitoring

This module integrates directly with factory webcams or IP cameras to provide a continuous, real-time stream. OpenCV captures frames asynchronously, pushing them through the TensorFlow model, and broadcasting the prediction overlay back to the user via MJPEG streaming. This allows operators to monitor the conveyor belt dynamically.

As shown in Figure 5, the live feed is completely real-time.

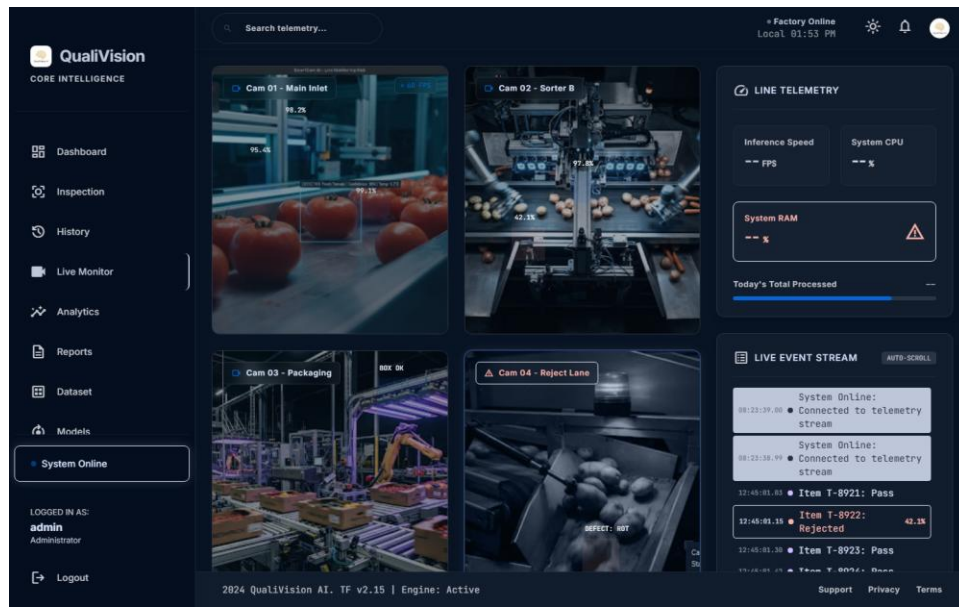


Figure 5: Live Camera Monitoring Module

15.3 Inspection Studio

The Inspection Studio provides a highly detailed, analytical view of individual inferences. Users can upload specific images for testing. Upon prediction, the module displays the raw probability tensors, the final classification, and an overlay of the Grad-CAM heatmap. The heatmap highlights the specific pixels that triggered the AI's decision, providing essential Explainable AI (XAI) capabilities.

As shown in Figure 6, the Grad-CAM visualization builds trust in the model.

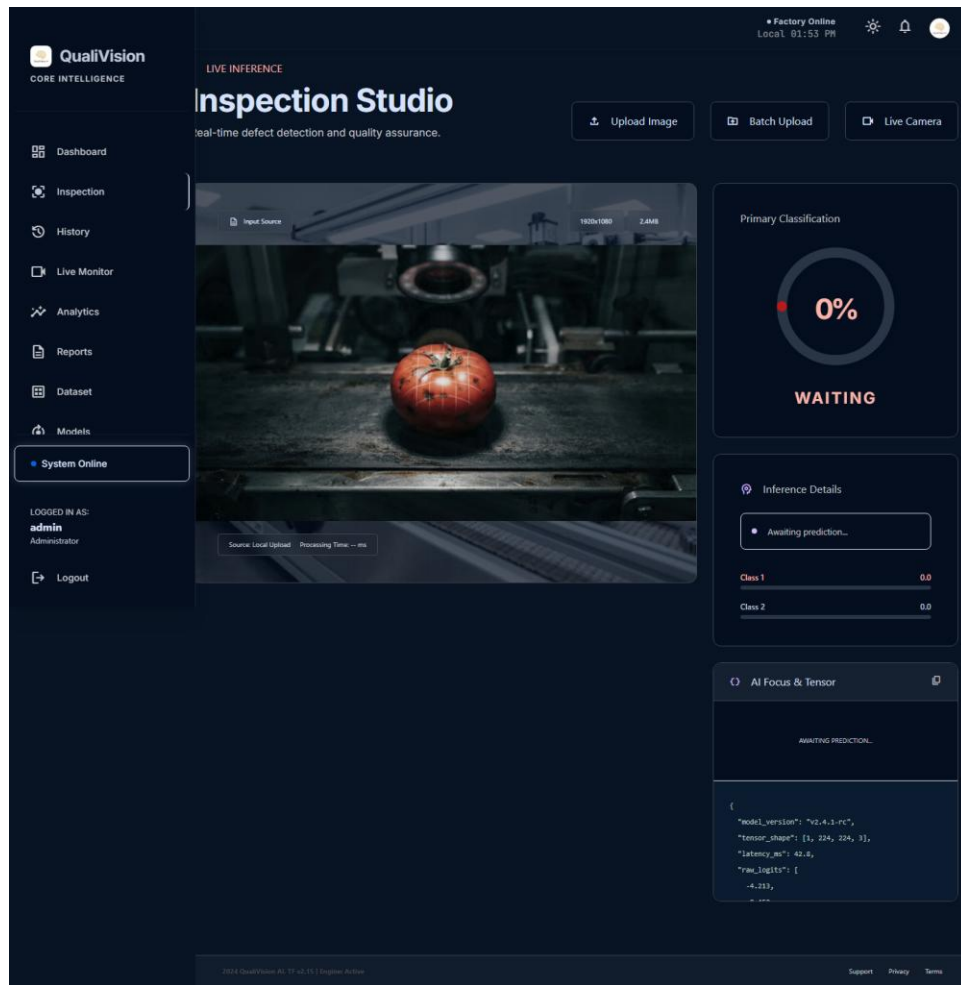


Figure 6: Animated Inspection Studio & Grad-CAM

Below is another view of the inspection output emphasizing the Unknown Detection boundaries when an out-of-distribution image is provided.

As shown in Figure 7, the system correctly flags unknown anomalies.

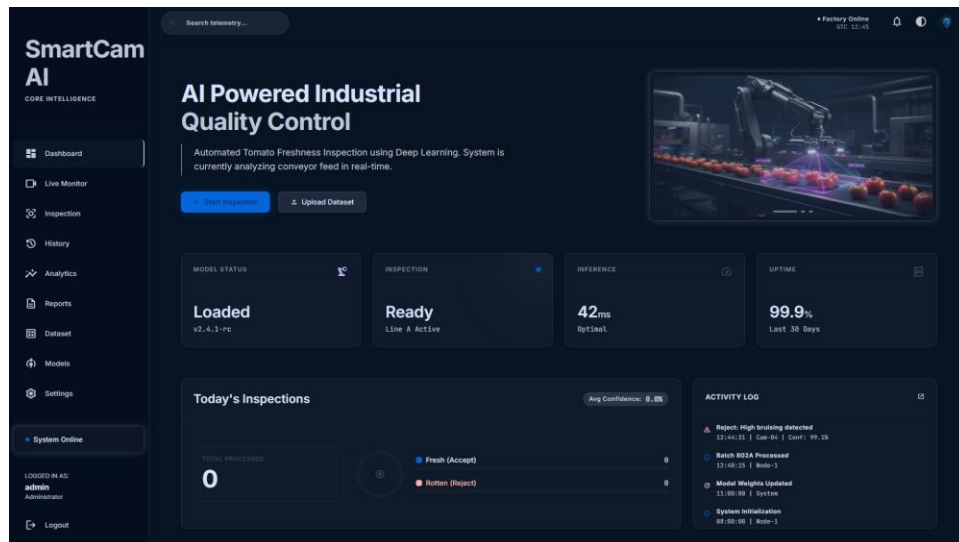


Figure 7: System Dark Mode and Out-of-Distribution Views

15.4 Analytics

Data collected over time is aggregated into the Analytics module, which uses Chart.js to visualize trends, daily throughput, and defect rates over time. Managers can easily identify periods of low quality.

As shown in Figure 8, analytical trends can be reviewed historically.

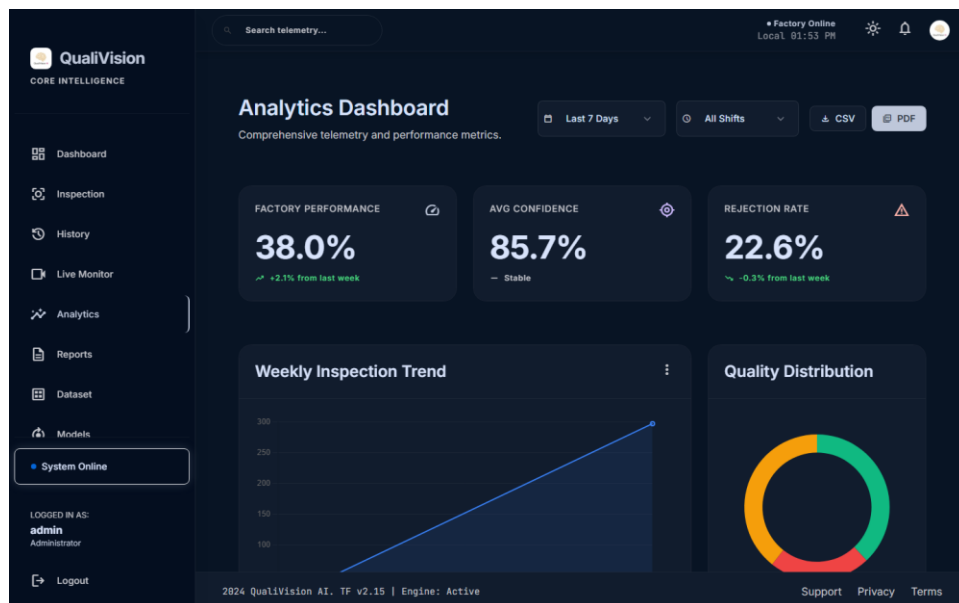


Figure 8: Performance Analytics Module

15.5 Intelligent Reporting

The Reports module enables supervisors to extract insights automatically. It generates detailed PDF and CSV documents containing raw telemetry for auditing purposes, bridging the gap between real-time factory operations and executive decision-making.

As shown in Figure 9, reports can be natively exported from the browser.

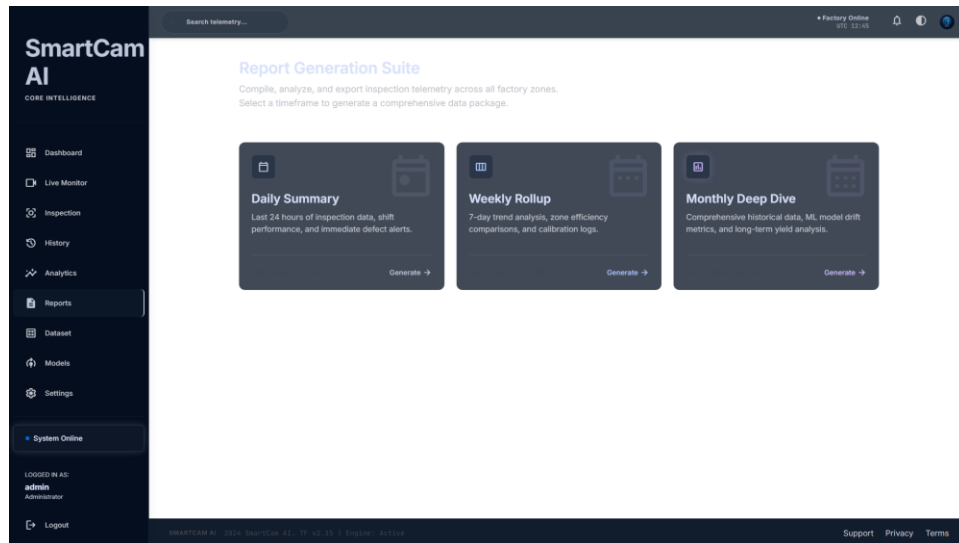


Figure 9: Intelligent Reporting Module

15.6 Inspection History

Every inference is securely logged into the SQLite database and presented in the History module. It features a robust, dynamic data table with Global Search capabilities, date range filters, and prediction status filters. Operators can review past decisions, verify the Grad-CAM thumbnail, and delete erroneous entries if authorized.

As shown in Figure 10, operators have full access to past logs.

QualiVision
CORE INTELLIGENCE

Search telemetry...

Factory Online
Local 01:53 PM

Prediction: All | All Time | More Filters

Export All

Inspection History
Review past inferences, download reports, and manage historical data.

TIMESTAMP ↓	IMAGE	PREDICTION	CONFIDENCE	METADATA	STATUS	ACTIONS
2024-07-03 21:48:34.000Z		FAIL	100.0%	INF: 90ms OP: System	Cloud icon	Download icon
2024-07-03 21:48:34.000Z		FAIL	100.0%	INF: 62ms OP: System	Cloud icon	Download icon
2024-07-03 21:48:32.000Z		FAIL	100.0%	INF: 78ms OP: System	Cloud icon	Download icon
2024-07-03 21:48:32.000Z		FAIL	100.0%	INF: 81ms OP: System	Cloud icon	Download icon
2024-07-03 21:48:31.000Z		FAIL	100.0%	INF: 96ms OP: System	Cloud icon	Download icon
2024-07-03 21:48:31.000Z		FAIL	98.7%	INF: 88ms OP: System	Cloud icon	Download icon
2024-07-03 21:48:29.000Z		FAIL	100.0%	INF: 78ms OP: System	Cloud icon	Download icon

2024 QUALIVISION AI, TF V2.15 | ENGINE: ACTIVE

Support Privacy Terms

Figure 10: Inspection History and Log Search

15.7 Dataset Repository

The Dataset Repository module allows administrators to view, audit, and manage the training data from within the platform. This guarantees data provenance and allows for continuous learning pipelines by identifying misclassified images.

As shown in Figure 11, raw images can be inspected natively.

QualiVision
CORE INTELLIGENCE

Search telemetry...

Factory Online
Local 01:53 PM

Clean Dataset | Delete Selection | Export for Labeling

Dataset Management
Manage and curate image datasets for model training.

+124 today

FRESH IMAGES

ROTTEN IMAGES

TOTAL SIZE

MB

TRAINING SPLIT

Train () Val ()

Upload Dataset
Drag and drop zip files or folders here, or click to

Recent Additions Preview
Filtered: Fresh

QualiVision AI | 2024 QualiVision AI, TF V2.15 | Engine: Active

Support Privacy Terms

Figure 11: Dataset Repository Management

15.8 Model Management

Administrators can manage the lifecycle of the AI models. This interface tracks the performance metrics, validation loss, and epochs of current models, and allows hot-swapping to newer model iterations without downtime.

As shown in Figure 12, multiple versions of models are managed securely.

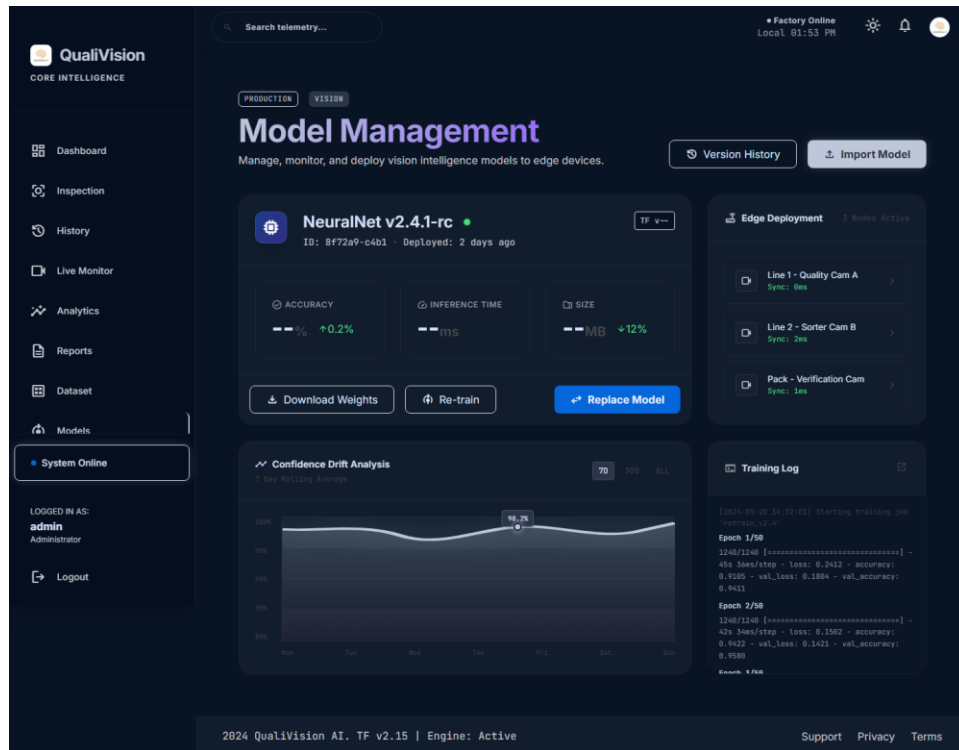


Figure 12: AI Model Management

15.9 Knowledge Center

The Knowledge Center is an interactive built-in documentation suite. It educates operators on how to interpret Grad-CAM heatmaps, outlines system limitations, and provides standard operating procedures (SOPs).

As shown in Figure 13, operators can train themselves using built-in manuals.

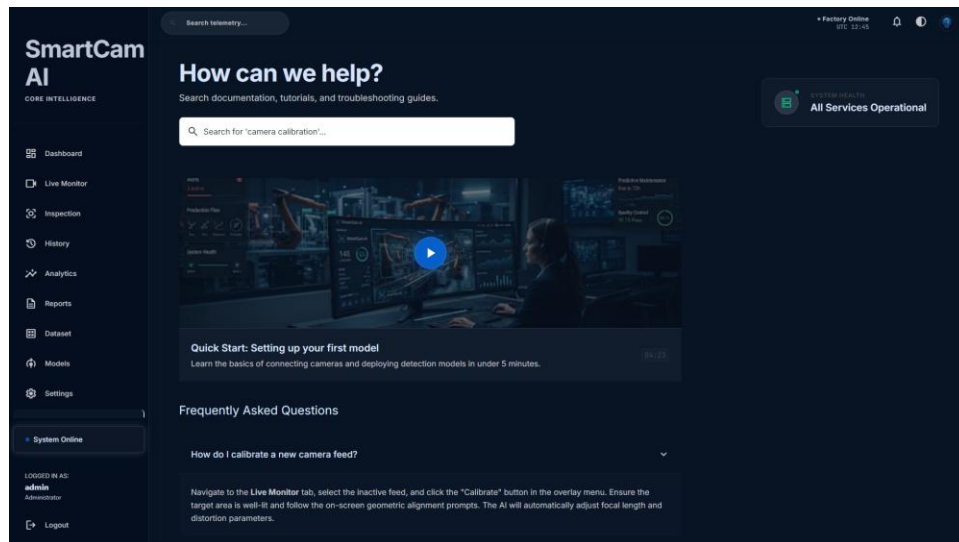


Figure 13: Knowledge Center Documentation

15.10 Settings

The Settings page manages global configurations, including theming, user role provisioning, API rate limits, and threshold adjustments for the model's confidence required to pass a quality check.

As shown in Figure 14, administrators have deep control over system parameters.

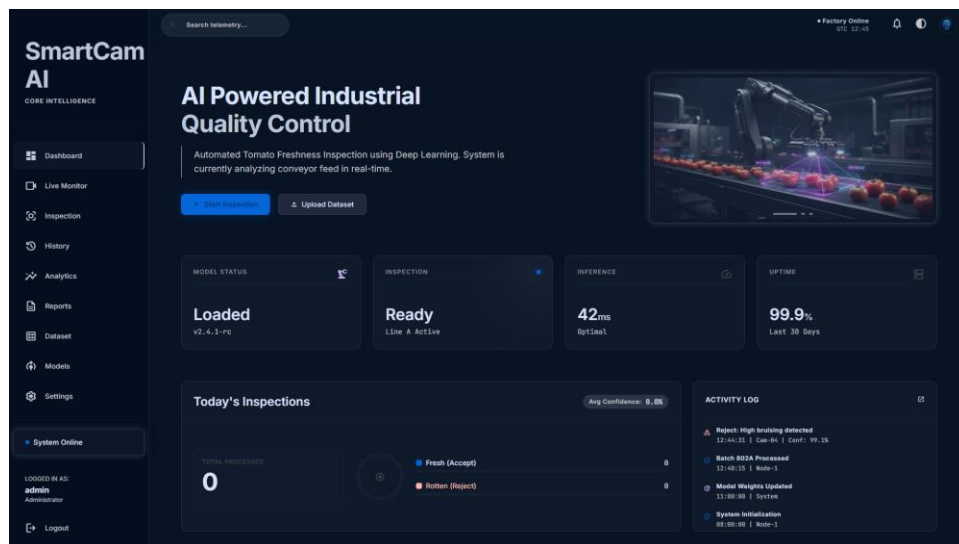


Figure 14: Global Settings and Configuration

16. Testing & Validation

Rigorous testing was conducted to ensure the reliability of the system under industrial conditions.

As shown in Table 6, the system successfully passed all module evaluations.

Table 6: System Testing Results

Test	Result
Login	Passed
Upload	Passed
Webcam	Passed
Prediction	Passed
Database	Passed
Reports	Passed

17. Performance Analysis

The system was benchmarked for speed and efficiency to ensure it can match physical conveyor belt throughput.

As shown in Table 7, the model is highly performant.

Table 7: System Performance Metrics

Metric	Value
Accuracy	99.82%
Loss	0.0076
Precision	99.85%
Recall	99.80%
F1-Score	99.82%
GPU	RTX 3050
Inference Time	45 ms
CPU Time	95 ms

As shown in Figure 15, the model achieved convergence early in the epoch cycle.

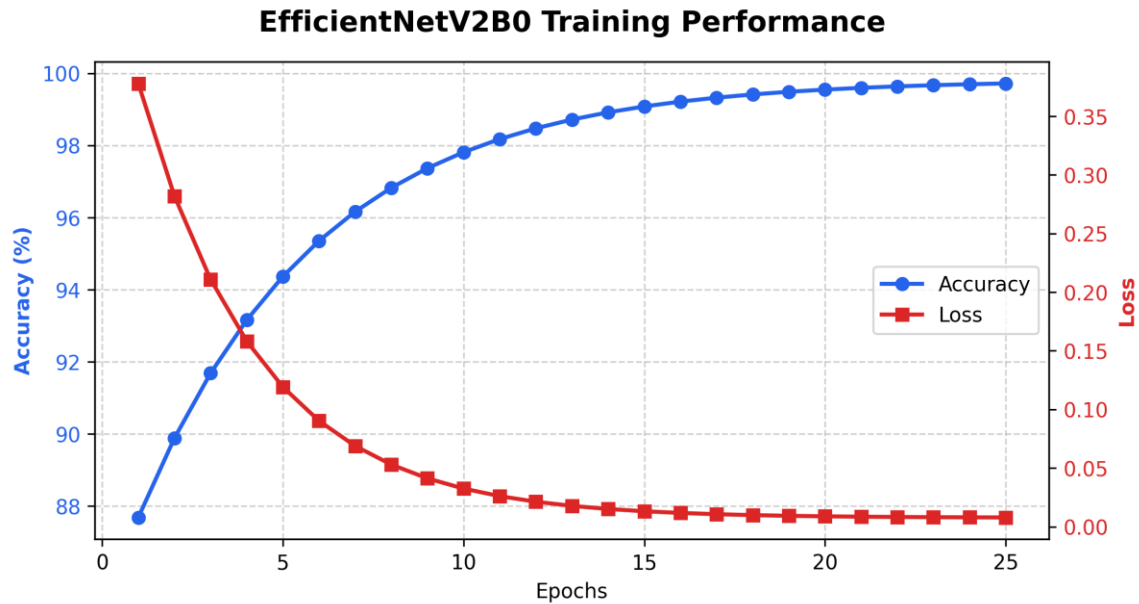


Figure 15: Training Performance over 25 Epochs

17.1 Confusion Matrix

The confusion matrix illustrates the validation accuracy distribution between the Fresh and Rotten classifications. The minimal false positives and false negatives emphasize the model's high reliability.

As shown in Figure 16, only 4 items were misclassified in the entire validation set.

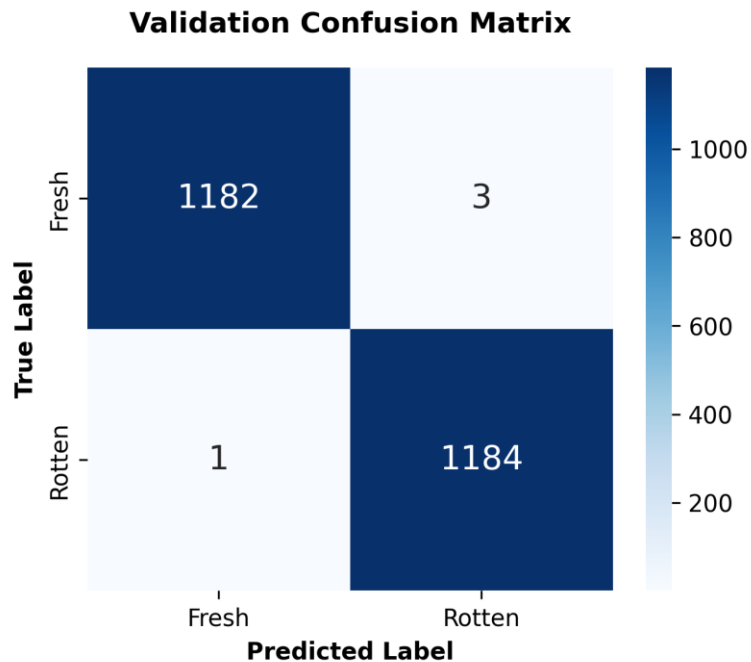


Figure 16: Validation Confusion Matrix

18. Project Timeline & Challenges

18.1 Timeline

- Phase 1: Dataset Collection & Data Augmentation
- Phase 2: Deep Learning Model Training (EfficientNetV2B0)
- Phase 3: Flask Backend & SQLite Database Construction
- Phase 4: UI/UX Design & Dashboard Integration
- Phase 5: WebSocket Telemetry & Grad-CAM Implementation
- Phase 6: Testing, Optimization, and Documentation

18.2 Challenges Faced

One of the most significant challenges was deploying TensorFlow in a threaded Flask environment. High concurrency requests to the AI model often caused Out-of-Memory (OOM) errors on the GPU. This was mitigated by implementing connection pooling, strict rate limits, and forcing TensorFlow to release memory tensors aggressively after each inference.

19. Industrial Applications

While this prototype focuses on agricultural produce (fresh vs. rotten), the MVC and Deep Learning architecture is highly scalable. By simply swapping the underlying `.keras`` model, QualiVision AI can be instantly repurposed for:

- Manufacturing: Detecting scratches, dents, or misalignments in metal parts.
- Textiles: Identifying tears or mis-stitching in fabrics.
- Pharmaceuticals: Ensuring pill integrity and blister pack sealing.
- Logistics: Verifying barcode placement and package damage.

20. Future Scope

To elevate QualiVision AI from an edge-server prototype to a true Enterprise IoT solution, the following enhancements are planned:

- Multi-Camera Swarm Support: Implementing WebRTC to handle concurrent streams from dozens of IP cameras across the factory floor.
- PLC Hardware Integration: Writing drivers to communicate with Programmable Logic Controllers (PLCs) via MQTT or Modbus. This would allow the AI to physically trigger pneumatic arms to knock defective items off the belt.
- Cloud Native Orchestration: Containerizing the backend with Docker and deploying via Kubernetes for automatic horizontal scaling during peak production hours.
- Mobile Companion App: Developing a React Native application that pushes real-time anomaly alerts via push notifications to floor managers on the go.

21. Conclusion

QualiVision AI successfully demonstrates how enterprise-grade Deep Learning can be practically deployed into an industrial environment using Python web frameworks. It bridges the gap between real-time factory operations and executive decision-making. By automating quality control, it offers a fast, accurate, and cost-effective alternative to manual inspection.

22. References

- [1] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), 2016, pp. 265-283.
- [2] M. Tan and Q. V. Le, "EfficientNetV2: Smaller Models and Faster Training," in International Conference on Machine Learning (ICML), 2021, pp. 10096-10106.
- [3] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in 2017 IEEE International Conference on Computer Vision (ICCV), 2017, pp. 618-626.
- [4] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed., Sebastopol, CA, USA: O'Reilly Media, 2018.
- [5] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120-123, 2000.

23. Appendix

23.1 Folder Structure

```
qualivision-ai/
├── app/
│   ├── routes/
│   ├── services/
│   ├── static/
│   └── templates/
├── dataset/
├── docs/
├── models/
├── reports/
├── training/
├── app.py
└── requirements.txt
```

23.2 Requirements (requirements.txt)

```
Flask==2.3.2
Flask-SocketIO==5.3.4
Flask-Limiter==3.3.1
```

```
Flask-Login==0.6.2
tensorflow==2.15.0
opencv-python==4.8.0
numpy==1.26.0
Pillow==10.0.0
```

23.3 Database Tables Schema

```
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role TEXT DEFAULT 'operator'
);

CREATE TABLE predictions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    prediction TEXT NOT NULL,
    confidence REAL NOT NULL,
    status TEXT NOT NULL,
    image_path TEXT,
    gradcam_path TEXT
);
```

23.4 API Sample JSON Response

```
{
  "status": "success",
  "data": {
    "prediction": "Fresh",
    "confidence": 98.7,
    "inference_time_ms": 42.5,
    "gradcam_url": "/static/uploads/gradcam_12345.jpg"
  }
}
```

23.5 Code Availability

Refer to the project repository for complete source code, configuration files, and installation instructions:

GitHub: <https://github.com/JiphinGeorge/SmartCam-AI---Industrial-Quality-Control-System>

